

Report - Assignment 3

Date of submission : 06/10/24

*Vaibhav Ahuja
Pratyush Agarwal
Aryan Verma*

Objective function -

1. **Final Submission** - Used *CrossEntropy* loss
2. The selected model (**ResNet**) are suited for CrossEntropy loss, as said in the paper, hence went for it. Tried out MSE loss also, the results were not very satisfactory
3. Reasons why *CrossEntropy* loss was best suited -
 - a. Outperforms other optimization techniques used with dense net (**source - [1]**)
 - b. Highly effective for multi-class classification tasks like CIFAR-100. The cross-entropy loss function works by measuring the difference between the predicted probability distribution and the true class label.

Model Architectures

We experimented with several architectures, including ResNet-18[3], ResNet-50[3], WideResNet-50_2, EfficientNet-B0, [5]DenseNet-121, 169, and 201. After testing these models, we selected **DenseNet-121** as the final architecture due to its superior performance and efficiency.

- **DenseNet-121** was chosen because it:
 - Promotes feature reuse through dense connections, ensuring efficient use of parameters and better gradient flow.
 - Has fewer parameters than deeper models like ResNet-50 or DenseNet-169, while still achieving high accuracy, making it suitable for constrained resources.
 - Demonstrated faster convergence and produced more reliable, high-confidence predictions.

DenseNet-121's balance between parameter efficiency and accuracy made it the optimal choice for this project.

Model Details-

The DenseNet-121 architecture is a type of convolutional neural network (CNN) known for its efficiency and ability to facilitate **feature reuse** through dense connections. Each layer in DenseNet receives inputs from all preceding layers, leading to enhanced gradient flow and a **reduction in the vanishing gradient problem**, which is particularly advantageous in deep networks.

1. **Input Layer:** The model begins with a convolutional layer (**conv0**), which operates on 3-channel input images (32x32x3). This layer uses a 3x3 kernel and 64 filters, followed by batch normalisation (**norm0**), a ReLU activation function (**relu0**), and an identity mapping for pooling (**pool0**).
2. **Dense Blocks:** The architecture contains four dense blocks. Each block comprises multiple dense layers that are connected in a feed-forward manner, where each layer takes input from all preceding layers. This allows for a high degree of feature reuse, making the model both parameter-efficient and capable of learning complex patterns with fewer layers.
 - Each dense layer includes a pair of convolutional operations:
 - A 1x1 convolution that increases the channel depth.
 - A 3x3 convolution that extracts feature maps, followed by batch normalisation and ReLU activation.
 - As layers are added, the depth of the feature maps increases, ensuring that the model captures diverse representations.
3. **Transition Layers:** After each dense block, transition layers reduce the spatial dimensions of the feature maps using 1x1 convolutions and average pooling. These layers play a critical role in controlling model complexity and preventing the feature maps from becoming overly large.
4. **Final Layers:** Following the fourth dense block, the network passes through a final batch normalisation layer (**norm5**) before the classifier. The classifier is a fully connected layer that outputs predictions for 100 classes, making this architecture suitable for CIFAR-100 classification tasks.
5. **Key Strengths:**
 - **Dense Connections:** These connections facilitate efficient feature reuse, reducing the number of parameters and improving gradient propagation.
 - **Batch Normalization and ReLU Activation:** These layers after every convolution ensure stable training and faster convergence by normalizing the activations and introducing non-linearity.

DenseNet-121 is highly regarded for its balance between model depth and efficiency, making it a great choice for tasks where computational resources are limited but accuracy is still a priority.

Summary of Dense Block I/O:

1. **Dense Block 1:**
 - Input: 64 channels.
 - Output: 256 channels (6 layers).
2. **Dense Block 2:**
 - Input: 128 channels.
 - Output: 512 channels (12 layers).
3. **Dense Block 3:**
 - Input: 256 channels.
 - Output: 1024 channels (24 layers).
4. **Dense Block 4:**
 - Input: 512 channels.
 - Output: 1024 channels (16 layers).

Detailed Flow:

1. **Initial Conv Layer:** 3-channel image $\rightarrow$ 64 channels.
2. **Dense Block 1:** 64 $\rightarrow$ 256 channels (6 layers).
3. **Transition 1:** 256 $\rightarrow$ 128 channels.
4. **Dense Block 2:** 128 $\rightarrow$ 512 channels (12 layers).
5. **Transition 2:** 512 $\rightarrow$ 256 channels.
6. **Dense Block 3:** 256 $\rightarrow$ 1024 channels (24 layers).
7. **Transition 3:** 1024 $\rightarrow$ 512 channels.
8. **Dense Block 4:** 512 $\rightarrow$ 1024 channels (16 layers).
9. **Final BatchNorm + Classifier:** 1024 $\rightarrow$ 100 classes.

Justification of Approach and Experiments

Why Dense Net 121 :

- We began by testing several well-known deep learning architectures that have been successful in image classification tasks, including **ResNet-18**, **ResNet-50**, **WideResNet-50_2**, **EfficientNet-B0**, and **DenseNet** models (121, 169, 201).
- **ResNet-18** and **ResNet-50**: These architectures are known for their residual connections, which help avoid the vanishing gradient problem. While ResNet-18 performed adequately with faster training times, it did not achieve the desired accuracy on CIFAR-100 due to its shallower depth (Lesser predictions for threshold and more incorrect predictions). ResNet-50

improved performance but at the cost of increased parameters and longer training times which also made the model overfit to the training data (hence changed the model to find if there is a better alternative).

- **WideResNet-50_2**: This variation with a wider architecture showed better results than ResNet-18 due to its higher capacity, but it still had issues with overfitting, which was reflected in the larger gap between training and validation accuracies.
- **EfficientNet-B0**: EfficientNet-B0, known for its high efficiency, provided faster convergence due to lesser parameters but accuracy was not very good.

Outcome: After comparing these models, **DenseNet-121** provided the best balance between performance, parameter efficiency, and training time. Its dense connections promote feature reuse, enabling better gradient flow and learning with fewer parameters, making it well-suited for our task.

Feasibility of Dense Net 121 :

Enhancing Information Transmission: Traditional convolutional networks connect layers sequentially, with each layer receiving input only from the previous one, which can limit information flow and lead to issues like the vanishing gradient problem in deeper networks. Dense blocks address this by creating direct connections between each layer and all subsequent layers, allowing every layer to access feature maps from all previous ones, thus improving feature representation and gradient flow across the network.

Mitigating the Vanishing Gradient Issue: As networks grow deeper, gradient signals can diminish, slowing learning and hindering training. Dense blocks address this by providing multiple direct paths for gradients to propagate, allowing them to flow freely across the network. This ensures deeper layers benefit from gradients originating earlier, making training very deep networks more manageable without the vanishing gradient problem.

Promoting Feature Map Reuse: In conventional architectures, each layer learns a distinct set of features, often leading to redundancy as similar patterns are relearned by different layers. Dense blocks eliminate this redundancy by reusing the feature maps from all preceding layers. Each layer can, therefore, build upon the collective knowledge of earlier layers, making the network more efficient in terms of parameters while avoiding unnecessary repetition in feature learning.

Parameter Efficiency: Although dense connections may seem like they would increase the number of parameters, they actually lead to more efficient use of them. By reusing feature maps from prior layers, the network avoids the need to relearn redundant features, thus reducing the overall number of parameters. Consequently, DenseNets are more compact and require fewer parameters compared to architectures like ResNet, all while maintaining or even improving performance.

Implicit Deep Supervision: Dense blocks also provide a form of inherent deep supervision. Since each layer can directly access both the original input and the gradients from the loss function, it helps each layer learn meaningful representations. This implicit supervision ensures that each layer is better guided during the training process, especially as the network becomes deeper, leading to more effective learning in deeper layers without solely relying on intermediate transformations.

Facilitating Gradient Flow and Simplifying Training: The dense connectivity structure improves the flow of gradients throughout the network, leading to more stable training and faster convergence. With multiple pathways for information to travel, gradients from the loss function can reach earlier layers more easily, making the network easier to optimise compared to other very deep architectures. This smoother gradient flow results in more efficient training overall

Data Preparation

1. Preprocessing Methods:

- **Normalisation:**

- The images are normalised using the mean and standard deviation of the CIFAR-100 dataset. This ensures that each channel of the image has a zero mean and unit variance, which helps in stabilising the training process and ensures faster convergence.

2. Augmentation Techniques:

- **Random Crop with Padding:**

- A padding of 4 pixels is added on each side of the image, increasing the size from 32×32 to 40×40 . From this padded image, a random crop is taken to bring it back to 32×32 . This technique helps make the model more robust to small shifts and distortions in the data.

- **Random Horizontal Flip:**

- The image is randomly flipped horizontally with a 50% probability. This technique introduces variance into the training data and helps the model generalise better to flipped versions of the same image, making it invariant to horizontal flips.

3. Dataset Splitting Strategy - Training and Validation Splits:

- In this context, the CIFAR-100 dataset is split into:

- **Training Set:** Contains the majority of the data (**90%** of the dataset) used for learning.
- **Validation Set:** A smaller portion of the data (**10%** of the dataset) held out to assess the model's performance during training to monitor overfitting. The validation set was **balanced** to have equal images from each class

Training Methodology

1. Model Architecture:

- We used a standard deep learning architecture suitable for image classification tasks. The model was trained on the CIFAR-100 dataset, and its complexity was chosen based on the capacity required to achieve high accuracy for this multi-class classification task.

2. Hyperparameters:

- **Batch Size:** We used a batch size of 128, which provided a good balance between convergence speed and memory utilisation.
- **Learning Rate:** Several learning rates were tested, ranging from $1e-3$ to $1e-4$. The best results were obtained with a learning rate of $5e-4$, providing stable convergence without overshooting.
- **Epochs:** The total number of training epochs was set to 60. Early stopping was applied to prevent overfitting by halting the training if the validation performance did not improve within a set patience threshold.

3. Loss Function Optimization:

- **Loss Optimization:** The learning process involves monitoring the training and validation loss to ensure proper convergence. It was optimised using stochastic gradient descent (SGD) with momentum for better convergence. We adjusted the learning rate based on validation performance, and used early stopping to terminate training when the loss did not decrease further, preventing unnecessary overfitting and reducing computation time.

4. Regularisation Techniques:

- **L2 Regularization (Weight Decay):** L2 regularisation with a penalty of $\lambda=5e-4$ was applied to the model. This technique helps prevent overfitting by penalising large weight values. L2 regularisation was particularly useful in this case, as we observed a more significant than expected difference between training and testing accuracies.
- **Early Stopping:** To prevent overfitting and reduce computation time, early stopping was used. The model training was halted when the validation loss stopped improving, ensuring we only kept the best-performing model.

Inference :

* Prediction Methodology: Describe the inference process, detailing how predictions are generated and any postprocessing steps applied. * Confidence Threshold: If applicable, explain how a confidence threshold was used to filter predictions, including reasoning behind the chosen threshold value.

Inference Process:

- During the inference phase, each test image is first preprocessed using a set of transformations (e.g., normalization with dataset-specific means and standard deviations).
- The preprocessed images are then passed through the modified DenseNet121 model to generate output logits, which represent unnormalized class scores.
- These logits are converted into class probabilities using the softmax function, ensuring that the sum of the probabilities over all classes equals 1.

Post-Processing:

- Once the probabilities are computed, I apply a confidence thresholding mechanism. The model selects the top two class probabilities for each image. If the highest probability exceeds a confidence threshold of 0.9, the class corresponding to this probability is chosen as the predicted label.
- If the highest probability is below 0.9, I set the prediction to **-1**, indicating that the model is not confident enough to make a prediction for that sample. This thresholding reduces the chance of low-confidence or incorrect predictions affecting the model's performance.
- This approach is particularly useful in scenarios where incorrect predictions can have significant consequences, allowing us to only retain highly reliable classifications.

Confidence Threshold:

- The confidence threshold of 0.9 was not arbitrarily chosen but rather tuned through cross-validation on the trained model. I evaluated several threshold values during the validation process, and 0.9 was found to strike a good balance between retaining enough predictions and ensuring a high level of certainty in those predictions.
- The reasoning behind choosing 0.9 is as follows:

- **Higher Accuracy:** Predictions below 0.9 had a higher chance of being incorrect, so filtering them out led to an overall improvement in model accuracy.
- **Reducing False Positives:** By setting a relatively high threshold, we ensure that only predictions with a strong confidence level are considered, minimising the likelihood of false positives.
- **Empirical Results:** Through cross-validation, I observed that a threshold of 0.9 provided the best trade-off between accuracy and prediction coverage.

Cross-Validation Process:

- Cross-validation was performed using a **Stratified Shuffle Split** method to ensure the validation sets maintained the same class distribution as the training data. The dataset was split into training and validation sets, with 4000 examples set aside for validation (10% of the total data).
- The validation process involved evaluating different confidence thresholds (ranging from 0.7 to 0.95) and selecting the one that optimised model performance on unseen data.
- I specifically monitored the model's ability to maintain high precision (correctness of predictions) and recall (ability to make predictions on most samples). After this tuning process, 0.9 emerged as the most effective threshold.

Batch Size and Performance Optimization:

- The model was trained using a batch size of 128, and the Adam optimizer was used with a learning rate of $5e-4$ and weight decay of $5e-4$ for regularisation. These hyperparameters were chosen to prevent overfitting and to ensure smooth convergence.
- To improve performance during training and inference, I leveraged **DataLoader** optimizations, such as `pin_memory` and multi-threading (`num_workers=4`). This allowed for faster data loading, ensuring that the GPU remains fully utilised during training.
- The model training loop includes early stopping to save the best model based on validation loss, further ensuring that overfitting is minimised, and the best generalising model is used for prediction.

References -

- [1]<https://www.sciencedirect.com/science/article/pii/S2667096821000136/pdf?md5=158ada0135991ef5bb0b940fd7083a57&pid=1-s2.0-S2667096821000136-main.pdf>
- [2]<https://www.ncbi.nlm.nih.gov/pmc/articles/PMC9060995/>
- [3]https://www.cv-foundation.org/openaccess/content_cvpr_2016/papers/He_Deep_Residual_Learning_CVPR_2016_paper.pdf
- [4]<https://arxiv.org/pdf/2201.03545>
- [5]<https://arxiv.org/pdf/1608.06993>
- [6]<https://github.com/liuzhuang13/DenseNet>